```
`timescale 1ns / 1ps
`include "parameters.vh" // Including the parameters file

module csr_ssm #(parameter OPCODE_WIDTH = 8, parameter TRAP_ADDRESS = 0)
(
    input wire i_clk, i_rst_n,

    // Interrupts
    input wire i_external_interrupt,
    input wire i_software_interrupt,

    // Exceptions
    input wire i_is_inst_illegal,
    input wire i_is_ecall,
    input wire i_is_ebreak,
    input wire i_is_mret,

    // Instruction Misaligned Exception
    input wire [OPCODE_WIDTH-1:0] i_opcode,
    input wire [31:0] i_y,

    // CSR instruction
    input wire [2:0] i_funct3,
    input wire [11:0] i_csr_index,
    input wire [31:0] i_imm,
    input wire [31:0] i_rs1,

    output reg [31:0] o_csr_out,

    // Trap Handler
    input wire [31:0] i_pc,
    input wire writeback_change_pc,
    output reg [31:0] o_return_address,
    output reg [31:0] o_trap_address,
    output reg o_go_to_trap_q,
    output reg o_return_from_trap_q
);

// Instruction types including AMOs
wire opcode_store, opcode_load, opcode_branch, opcode_jal, opcode_jalr,
opcode_system, opcode_amo;

assign opcode_store  = (i_opcode == `S);
assign opcode_load   = (i_opcode == `LD);
assign opcode_branch = (i_opcode == `B);
assign opcode_jal    = (i_opcode == `J);
```

```
assign opcode_jalr   = (i_opcode == `JR);
assign opcode_system = (i_opcode == `CSR);
assign opcode_amo    = (i_opcode == `AMO);

// CSR operation type
localparam CSRRW  = 3'b001,
           CSRRS  = 3'b010,
           CSRRC  = 3'b011,
           CSRRWI = 3'b101,
           CSRRSI = 3'b110,
           CSRRCI = 3'b111;

// CSR register mapping
localparam MEPC = 12'h341,
           MSTATUS = 12'h300,
           MIE     = 12'h304,
           MTVEC   = 12'h305;

// Internal registers
reg [31:0] csr_in, csr_data;
wire csr_enable = opcode_system && i_funct3 != 0 && !writeback_change_pc;
reg [1:0] new_pc = 0;
reg go_to_trap, return_from_trap;
reg is_interrupt, is_exception, is_trap, update_enable;

// CSR registers
reg [31:0] mepc, mtvec;
reg mstatus_mie;
reg mie_meie;
reg mip_meip;

// Initialize registers to prevent undefined states
initial begin
    o_go_to_trap_q = 0;
    o_return_from_trap_q = 0;
    o_csr_out = 0;
    o_return_address = 0;
    o_trap_address = 0;
    mstatus_mie = 0;
    mie_meie = 0;
    mtvec = TRAP_ADDRESS;
    mepc = 0;
    mip_meip = 0;
    go_to_trap = 0;
    return_from_trap = 0;
    is_interrupt = 0;
```

```
    is_exception = 0;
    is_trap = 0;
    update_enable = 0;
end

// CSR Read Logic - Assigns correct data to `csr_data`
always @* begin
    case (i_csr_index)
        MEPC:    csr_data = mepc;
        MSTATUS: csr_data = {31'b0, mstatus_mie};
        MIE:     csr_data = {31'b0, mie_meie};
        MTVEC:   csr_data = mtvec;
        default: csr_data = 32'b0;
    endcase
end

// CSR Write Logic (Clocked)
always @(posedge i_clk or negedge i_rst_n) begin
    if (!i_rst_n) begin
        mepc <= 0;
        mtvec <= TRAP_ADDRESS;
        mstatus_mie <= 0;
        mie_meie <= 0;
        mip_meip <= 0;
    end else if (csr_enable) begin
        case (i_funct3)
            CSRRW:  csr_in = i_rs1;
            CSRRS:  csr_in = csr_data | i_rs1;
            CSRRC:  csr_in = csr_data & ~i_rs1;
            CSRRWI: csr_in = i_imm;
            CSRRSI: csr_in = csr_data | i_imm;
            CSRRCI: csr_in = csr_data & ~i_imm;
            default: csr_in = csr_data;
        endcase

        case (i_csr_index)
            MEPC:    mepc <= csr_in;
            MSTATUS: mstatus_mie <= csr_in[0];
            MIE:     mie_meie <= csr_in[0];
            MTVEC:   mtvec <= csr_in;
        endcase
    end
end

// CSR Read Output
always @(posedge i_clk or negedge i_rst_n) begin
```

```
    if (!i_rst_n)
        o_csr_out <= 0;
    else if (csr_enable)
        o_csr_out <= csr_data;
end

// Trap Detection
always @* begin
    is_interrupt = mstatus_mie && mie_meie && mip_meip;
    is_exception = i_is_inst_illegal || i_is_ecall || i_is_ebreak;
    is_trap = is_interrupt || is_exception;
    go_to_trap = is_trap;
    return_from_trap = i_is_mret;
end

// Trap Handling Logic
always @(posedge i_clk or negedge i_rst_n) begin
    if (!i_rst_n) begin
        o_go_to_trap_q <= 0;
        o_return_from_trap_q <= 0;
        o_return_address <= 0;
        o_trap_address <= 0;
    end else begin
        if (go_to_trap && !o_go_to_trap_q) begin
            mepc <= i_pc;
            o_go_to_trap_q <= 1;
            o_trap_address <= mtvec;
        end
        if (return_from_trap) begin
            o_return_address <= mepc;
            o_return_from_trap_q <= 1;
        end else begin
            o_go_to_trap_q <= 0;
            o_return_from_trap_q <= 0;
        end
    end
end

endmodule
```